

7705交流260324

目前实现的功能

1. 使用streamlit实现的Web界面。CLI
2. 实现三个主要的分析agent（技术面、新闻、基本面）-->风险分析agent-->决策agent
3. 实现简单 agent_tools
4. 抓取新闻和yfinance数据的工具
5. 简单的回测评估

现有功能项目结构

```
COMP7705-Agent-Quant/
├── agentgraph/ #  workflow编排层 (LangGraph)
│   ├── orchestrator.py # 图构建、节点连接、主运行入口 (核心)
│   └── state.py # 全局状态定义 AgentState
├── agents/ # 各角色智能体
│   ├── market_analyst.py # 技术面分析师
│   ├── news_analyst.py # 新闻分析师
│   ├── fundamentals_analyst.py # 基本面分析师
│   ├── risk_analyst.py # 风险分析师
│   ├── PM.py # PM 最终决策
│   └── utils/
│   ├── agent_tools.py # 给LLM调用的数据工具 (@tool)
│   └── output_prase.py # PM输出结构化模型 (TradingDecision)
├── dataflow/ # 数据访问与服务层
│   ├── service.py # DataService, 统一数据入口
│   ├── portfolio_manager.py # 持仓与风控参数状态管理
│   └── providers/
│       ├── YFinance.py # 价格、指标、实时基本面
│       ├── news_google.py # 新闻抓取
│       ├── fundamentals_akshare.py # 历史基本面 (PIT)
│       └── macro_calendar.py # 宏观事件 (日历)
├── app.py # CLI入口
├── streamlit_app.py # Web UI入口 (展示、回测、导出)
├── run_web.sh # Web 启动脚本
├── requirements.txt # 依赖清单
├── properties.env # 环境配置 (API Key等)
└── test/ # 回测与评估脚本、样本数据
```

代码流程落地

1. IntelliFin_Assistant.run() 传入 ticker/date/current_position_pct

2. AgentState 初始化状态容器
3. 三个分析师节点并行运行，各自可调用工具取数据
4. risk_analyst 等待三份上游报告齐全后执行
5. PM_agent 综合四份报告，输出结构化决策
6. 返回最终状态（含 PM_report、Action、Target_position_pct）

计划升级的地方

目录示意图

```
context/  
  store.py           # ContextStore抽象接口  
  memory_store.py    # 内存实现（开发）  
  sqlite_store.py    # SQLite实现（本地持久化）  
  schema.py          # 上下文数据模型  
  
hitl/  
  models.py          # 审批请求/审批结果模型  
  service.py         # HITL业务流程  
  policies.py        # 哪些条件必须人工审批  
  
broker_mock/  
  models.py          # 订单/成交/持仓模型  
  engine.py          # 撮合引擎  
  gateway.py         # 对上层暴露统一券商接口  
  risk_checks.py     # 下单前风控检查  
  
channels/  
  init.py  
  whatsapp/          # whatsapp接入  
  telegram/          # telegram接入  
  api/  
  webhook.py         # 统一消息接受
```

添加数据源

问题

yfinance抓取失败则换去其他数据源，例如alpha vantage，考虑接入第三方API获取更多

改进目标

考虑整理数据源，将历史抓去数据按照类型存入数据库：

1. 新闻摘要与总结
2. 结构化整理的财报数据
3. 结构化整理的价格数据、技术指标

上下文管理

问题

目前的上下文都储存在字典的 `message` 中，只能给出单次建议查询，不利于持续的跟进和管理仓位。

技术规划

考虑存储 `state` 中的 `ticker/date/current_position_pct` 等信息，Agent之前给出的报告（协作报告、工具调用日志、之前给出的投资建议：目标仓位、理由等），通过 `sqlite` 存储在 `.db` 中

目标

1. 任一分析会话可被完整回放（输入 -> 各Agent报告 -> PM决策）
2. 可按 `ticker` 检索最近 `N` 次决策
3. 重启程序后历史记录仍可查询（SQLite）
4. 历史操作保存

人机交互

问题

当前 PM 输出后直接结束。真实流程中，人通常要在关键场景介入：

高风险动作（大幅加仓/反手）

信号冲突（技术看涨、基本面看跌）

数据质量不达标（新闻不足、指标异常）

实现目标

这里其实主要考虑半自动模式，考虑中长线交易而非短线投机，agent给出投资建议和执行投资，如果出现比较大的调仓则需要人工approve：

- 通过API接入Whats App，实现在手机上发送消息：总结今日市场动向，给出投资建议；给出模拟交易的日报；申请进行比较大的仓位调整。
- 人工可以进行同意、驳回、修改参数重新运行

接入券商模拟盘接口

问题

当前系统主要是给建议，没有形成回测、决策、模拟的闭环

实现目标

- 将分析决策变成API接口，券商模拟平台调用分析结果标准化交易API调用方式（接近真实券商）
- 在本地模拟下单、成交、持仓、资金变化
- 通过模拟的运营状况，修正完善策略
- 让回测/仿真与agent决策形成闭环

参考技术栈

FastAPI , moomoo OpenAPI , 同花顺 OpenAPI

实现示例

```
class BrokerGateway(Protocol):
    def get_account(self) -> dict: ...
    def get_positions(self) -> list[dict]: ...
    def place_order(self, order: dict) -> dict: ...
    def cancel_order(self, order_id: str) -> dict: ...
    def get_order(self, order_id: str) -> dict: ...
```

实现BUY/SELL, MARKET/LIMIT

问题

1. 是否可以申请相关预算购买Token
2. 验收标准需要到什么程度